

# Syntactic Monoid

Matemática Discreta II

## Integrantes

Alejandro Rodríguez Arciniegas  
Kevin Alexander Rodríguez Forero  
Jhonatan Bolívar Laverde  
Keynes Stephens Watson

## Universidad Nacional de Colombia

Facultad de Ingeniería · Ingeniería de Sistemas y Computación

Periodo académico 2026-1 · Julio de 2026

Repositorio: [github.com/Arax48/syntactic-monoid](https://github.com/Arax48/syntactic-monoid)



UNIVERSIDAD  
**NACIONAL**  
DE COLOMBIA

## Contenido

|                                                                                           |    |
|-------------------------------------------------------------------------------------------|----|
| 1. Resumen .....                                                                          | 4  |
| 2. Motivación.....                                                                        | 4  |
| 2.1 Objetivos .....                                                                       | 4  |
| 2.2 Relevancia para Ingeniería de Sistemas y Computación .....                            | 5  |
| 3. Problema abordado.....                                                                 | 5  |
| 4. Fundamentos teóricos.....                                                              | 6  |
| 4.1 Alfabetos, palabras y el monoide libre .....                                          | 6  |
| 4.2 Autómatas Finitos Deterministas .....                                                 | 6  |
| 4.3 Monoides, homomorfismos y congruencias .....                                          | 6  |
| Primer Teorema del Isomorfismo (monoides) .....                                           | 6  |
| 4.4 Convención de composición .....                                                       | 7  |
| 4.5 El monoide de transición $M(A)$ : construcción y propiedades .....                    | 7  |
| Lema 0 (separación de $\delta^*$ ) .....                                                  | 7  |
| Transformaciones inducidas por palabras.....                                              | 7  |
| Definición, cerradura y existencia .....                                                  | 8  |
| 4.6 El homomorfismo natural $\varphi$ .....                                               | 8  |
| 4.7 Núcleo de $\varphi$ , congruencia y clases de equivalencia.....                       | 8  |
| 4.8 Primer Teorema del Isomorfismo aplicado a las cadenas.....                            | 9  |
| 4.9 $\varphi$ y el grupo simétrico $S_n$ (profundización solicitada por el profesor)..... | 9  |
| Ejemplo A — paridad de unos, $M(A) = S_2$ .....                                           | 10 |
| Ejemplo B — módulo 3, $M(A)$ subgrupo cíclico de $S_3$ .....                              | 10 |
| Ejemplo C — cadenas que terminan en “01”, $M(A) \not\subseteq S_n$ .....                  | 10 |
| Ejemplo D — paridad conjunta de dos símbolos, $M(A) = S_2 \times S_2 \cong V_4$ .....     | 11 |
| 4.10 Invarianza respecto al AFD mínimo .....                                              | 11 |
| 5. Diseño de la solución y desarrollo realizado.....                                      | 12 |
| 5.1 Metodología.....                                                                      | 12 |
| 5.2 Arquitectura del sistema .....                                                        | 12 |
| 5.3 Decisiones de diseño relevantes .....                                                 | 13 |
| 5.4 Interfaz web interactiva.....                                                         | 13 |
| 6. Algoritmos, estructuras y modelos utilizados.....                                      | 14 |
| 6.1 Algoritmo de construcción de $M(A)$ .....                                             | 14 |
| 6.2 Complejidad .....                                                                     | 14 |
| 6.3 Estructuras de datos.....                                                             | 14 |
| 7. Resultados obtenidos.....                                                              | 15 |
| 7.1 Ejemplo 1: paridad de unos .....                                                      | 15 |

|                                                                       |                                      |
|-----------------------------------------------------------------------|--------------------------------------|
| 7.2 Ejemplo 2: cantidad de unos $\equiv 0$ (mód 3) .....              | 15                                   |
| 7.3 Ejemplo 3: cadenas que terminan en "01" .....                     | 16                                   |
| Núcleo (hasta $ w  \leq 3$ ) .....                                    | 16                                   |
| 7.4 Ejemplo 4: paridad conjunta de dos símbolos (grupo de Klein)..... | 16                                   |
| 7.5 Tabla comparativa y verificación mediante pruebas .....           | 16                                   |
| 8. Limitaciones y posibles mejoras.....                               | 17                                   |
| 8.1 Limitaciones actuales .....                                       | 17                                   |
| 8.2 Mejoras futuras.....                                              | <b>¡Error! Marcador no definido.</b> |
| 9. Conclusiones .....                                                 | 18                                   |
| 10. Referencias bibliográficas y técnicas.....                        | 19                                   |
| 10.1 Fuentes bibliográficas.....                                      | 19                                   |

# 1. Resumen

Este trabajo presenta un estudio integrado, teórico y computacional, de los Autómatas Finitos Deterministas (AFD) desde una perspectiva algebraica. Tomando como objeto central el **monoide de transición**  $M(A)$  asociado a un AFD  $A = (Q, \Sigma, \delta, q_0, F)$ , demostramos rigurosamente que la asignación  $\varphi : \Sigma^* \rightarrow M(A)$ ,  $\varphi(w) = f_w$ , con  $f_w(q) = \delta^*(q, w)$ , es un homomorfismo sobreyectivo de monoides cuyo núcleo  $\text{Ker}(\varphi)$  es una congruencia de monoide sobre  $\Sigma^*$ . El Primer Teorema del Isomorfismo proporciona la identificación canónica  $\Sigma^*/\text{Ker}(\varphi) \cong M(A)$ , interpretada como una clasificación algebraica de las palabras según su comportamiento en el autómata.

Junto con la fundamentación matemática, desarrollamos una herramienta en Python que valida y simula AFDs, construye  $M(A)$  por búsqueda en anchura (BFS), genera la tabla de Cayley, exhibe el núcleo y sus representantes mínimos, y produce reportes y visualizaciones (Graphviz, matplotlib), además de un sitio web interactivo autocontenido en un único archivo HTML. El sistema está respaldado por 149 pruebas automatizadas (pytest) en verde. Se estudian en detalle cuatro AFDs canónicos: paridad de unos, conteo módulo 3, cadenas que terminan en “01” y paridad conjunta de dos símbolos. En los tres primeros, el monoide resulta ser, respectivamente,  $\mathbb{Z}/2\mathbb{Z}$ ,  $\mathbb{Z}/3\mathbb{Z}$  y un monoide aperiódico no conmutativo de 5 elementos; en el cuarto, el grupo de Klein  $V_4 = \mathbb{Z}/2\mathbb{Z} \times \mathbb{Z}/2\mathbb{Z}$ .

Atendiendo a la observación del profesor sobre profundizar en la estructura de  $M(A)$  y la función  $\varphi$ , dedicamos la Sección 4.9 a caracterizar con precisión cuándo  $M(A)$  es subgrupo del grupo simétrico  $S_n$  y cuándo escapa de esa categoría, mostrando explícitamente la composición en  $M(A)$  en términos de  $S_n$  para cada uno de los ejemplos trabajados.

## 2. Motivación

La motivación de este trabajo surgió durante una clase de Teoría de la Computación, al preguntarnos si sería posible tender un puente con los temas que veníamos trabajando en Matemática Discreta II. La idea de fondo es sencilla: en un AFD, muchas cadenas distintas terminan llevando a un estado exactamente igual, sin importar desde dónde se empieza a leer. Por ejemplo, en un autómata que cuenta unos módulo 3, las cadenas 11 y 0110 dejan el autómata en el mismo lugar sea cual sea el estado de partida. Esto no es casualidad, sino la manifestación de una estructura algebraica —un monoide finito— escondida detrás de la función de transición.

A cada AFD se le puede asociar entonces un monoide finito cuya estructura interna (si es grupo, cuál grupo, si es abeliano, cíclico o aperiódico) dice algo cualitativo sobre el lenguaje que reconoce. El proyecto se enmarca así en el puente entre Matemática Discreta II (estructuras algebraicas, relaciones de equivalencia, teoremas de isomorfismo) y Teoría de la Computación (AFDs, lenguajes regulares), apoyándose en [1] para el aparato algebraico y en [2] para los autómatas.

### 2.1 Objetivos

- Formalizar el monoide de transición  $M(A)$  y demostrar sus propiedades de monoide (cerradura, asociatividad, identidad).

- Definir el homomorfismo natural  $\varphi : \Sigma^* \rightarrow M(A)$  y demostrar que efectivamente lo es, incluyendo su relación con el grupo simétrico  $S_n$ .
- Describir  $\text{Ker}(\varphi)$  como congruencia de  $\Sigma^*$ , exhibir las clases de equivalencia y aplicar el Primer Teorema del Isomorfismo.
- Implementar el aparato anterior en Python, con una suite de pruebas exhaustiva y ejemplos ejecutables, además de una interfaz web interactiva.
- Documentar los resultados en un formato académico defendible, con demostraciones completas.

## 2.2 Relevancia para Ingeniería de Sistemas y Computación

El enfoque algebraico permite ver propiedades de los lenguajes regulares —aperiodicidad, pertenencia a la jerarquía “star-free”, complejidad de la sintaxis del lenguaje— que el modelo puramente combinatorio de un AFD no deja ver a simple vista. Esta perspectiva es, en el fondo, la misma que sustenta técnicas usadas en el diseño de compiladores (análisis léxico), la verificación de modelos y la teoría de la complejidad de circuitos, y sirve como una introducción natural a la teoría algebraica de autómatas (semigrupos y monoides sintácticos).

## 3. Problema abordado

**Pregunta central:** ¿qué información de un AFD y del lenguaje regular que reconoce puede leerse directamente en la estructura algebraica de su monoide de transición  $M(A)$ ?

Este problema se descompone en tres preguntas técnicas que estructuran el desarrollo matemático del proyecto:

- ¿Es  $M(A)$  efectivamente un monoide bien definido, y bajo qué operación?
- ¿Existe un homomorfismo natural entre las cadenas de entrada y  $M(A)$ , y qué relación de equivalencia induce sobre  $\Sigma^*$ ?
- ¿Bajo qué condiciones  $M(A)$  es un grupo —en particular un subgrupo del grupo simétrico  $S_n$ — y bajo cuáles escapa de esa categoría, convirtiéndose en un monoide aperiódico?

El monoide de transición  $M(A)$  no aparece explícitamente en ninguno de los dos textos base del curso: es una construcción propia del proyecto que tiende un puente entre la maquinaria de monoides, composición de funciones y homomorfismos descrita en [1], y la definición de AFD y su función de transición  $\delta^*$  descrita en [2]. El resto del informe desarrolla la solución completa a este problema, tanto en su vertiente teórica (Sección 4) como computacional (Secciones 5 a 7).

## 4. Fundamentos teóricos

### 4.1 Alfabetos, palabras y el monoide libre

Un **alfabeto**  $\Sigma$  es un conjunto finito no vacío cuyos elementos llamamos símbolos o letras. Una **palabra** sobre  $\Sigma$  es una secuencia finita  $w = a_1a_2\dots a_n$  con cada  $a_i \in \Sigma$ . La longitud de  $w$  es  $|w| = n$ . La palabra de longitud cero es la **palabra vacía**  $\lambda$ .

Denotamos por  $\Sigma^*$  al conjunto de todas las palabras sobre  $\Sigma$ , incluyendo  $\lambda$ . La concatenación  $\cdot : \Sigma^* \times \Sigma^* \rightarrow \Sigma^*$  es asociativa y  $\lambda$  es su elemento neutro. Por tanto  $(\Sigma^*, \cdot, \lambda)$  es un monoide: el monoide libre generado por  $\Sigma$ . Es libre porque toda función  $\Sigma \rightarrow M$  hacia un monoide  $M$  se extiende de manera única a un homomorfismo de monoides  $\Sigma^* \rightarrow M$ . Un lenguaje sobre  $\Sigma$  es cualquier subconjunto  $L \subseteq \Sigma^*$ .

### 4.2 Autómatas Finitos Deterministas

Un AFD es una 5-tupla  $A = (Q, \Sigma, \delta, q_0, F)$  con:

- $Q$ : conjunto finito de estados.
- $\Sigma$ : alfabeto.
- $\delta : Q \times \Sigma \rightarrow Q$ : función de transición (total).
- $q_0 \in Q$ : estado inicial.
- $F \subseteq Q$ : conjunto de estados de aceptación.

La función de transición se extiende a palabras como  $\delta^* : Q \times \Sigma^* \rightarrow Q$  mediante:

$$\delta^*(q, \lambda) = q, \quad \delta^*(q, wa) = \delta(\delta^*(q, w), a) \quad \text{para todo } w \in \Sigma^*, a \in \Sigma.$$

El lenguaje aceptado por  $A$  es  $L(A) = \{ w \in \Sigma^* : \delta^*(q_0, w) \in F \}$ . Un lenguaje es regular si y sólo si es aceptado por algún AFD.

### 4.3 Monoides, homomorfismos y congruencias

**Monoide.** una tripla  $(M, *, e)$  con  $*$  operación binaria asociativa sobre  $M$  y  $e \in M$  elemento neutro a ambos lados.

**Homomorfismo de monoides.** una función  $h : M \rightarrow N$  entre  $(M, *, e)$  y  $(N, \otimes, e')$  que preserva la operación y el neutro:  $h(x * y) = h(x) \otimes h(y)$  y  $h(e) = e'$ .

**Congruencia de monoide.** una relación de equivalencia  $\sim$  sobre  $M$  compatible con la operación: si  $x \sim x'$  y  $y \sim y'$  entonces  $x*y \sim x'*y'$ . El conjunto cociente  $M/\sim$  hereda estructura de monoide, y la proyección canónica  $\pi : M \rightarrow M/\sim$  es un homomorfismo.

#### Primer Teorema del Isomorfismo (monoides)

Si  $h : M \rightarrow N$  es un homomorfismo de monoides, la relación  $x \sim_h y \Leftrightarrow h(x) = h(y)$  es una congruencia y existe un único isomorfismo  $\bar{h} : M/\sim_h \rightarrow \text{Im}(h)$  tal que  $h = \bar{h} \circ \pi$ .

La demostración es análoga a la versión para grupos que puede consultarse en [1]: se define  $\tilde{h}([x]) := h(x)$ , y se verifica buena definición, homomorfismo, inyectividad y sobreyectividad sobre  $\text{Im}(h)$ . Esta versión general se instancia en la Sección 4.8 para el caso concreto  $h = \varphi$ .

#### 4.4 Convención de composición

Como las palabras se leen de izquierda a derecha y las funciones se componen clásicamente de derecha a izquierda ( $(g \circ f)(q) := g(f(q))$ ), surge una posible confusión de orden al pasar de  $\Sigma^*$  a transformaciones sobre  $Q$ . Para evitarla fijamos, para todo el documento, la operación diagramática o de lectura izquierda-a-derecha sobre el conjunto  $Q^\varphi$  de funciones  $Q \rightarrow Q$ :

$$(f \cdot g)(q) := g(f(q)), \text{ es decir, } f \cdot g := g \circ f.$$

La tripla  $(Q^\varphi, \cdot, \text{id}_Q)$  es un monoide (el monoide opuesto del monoide funcional usual  $(Q^\varphi, \circ, \text{id}_Q)$ ), pues  $\cdot$  es asociativa e  $\text{id}_Q$  es neutro a ambos lados. Con esta convención, leer  $uv$  significa “primero aplicar  $f_u$ , luego  $f_v$ ”, igual que se compone con  $\cdot$ . En consecuencia,  $\varphi : \Sigma^* \rightarrow M(A)$  resultará ser un homomorfismo en el sentido estándar, sin signo opuesto ni anti-homomorfismo. Todas las apariciones de  $\cdot$  en este informe se entienden en esta convención.

#### 4.5 El monoide de transición $M(A)$ : construcción y propiedades

Fijamos un AFD  $A = (Q, \Sigma, \delta, q_0, F)$ . Antes de definir el monoide necesitamos un lema fundamental que no está explícito en la definición recursiva de  $\delta^*$ .

##### Lema 0 (separación de $\delta^*$ )

Para todo  $q \in Q$  y  $u, v \in \Sigma^*$ :  $\delta^*(q, uv) = \delta^*(\delta^*(q, u), v)$ .

##### Demostración.

Por inducción sobre  $|v|$ . Caso base  $v = \lambda$ :  $\delta^*(q, u\lambda) = \delta^*(q, u) = \delta^*(\delta^*(q, u), \lambda)$ , por la definición de  $\delta^*$  aplicada al lado derecho. Paso inductivo,  $v = wa$  con  $a \in \Sigma$  y  $|w| < |v|$ : asumiendo por hipótesis de inducción que  $\delta^*(q, uw) = \delta^*(\delta^*(q, u), w)$ ,

$$\delta^*(q, u(wa)) = \delta^*(q, (uw)a) = \delta(\delta^*(q, uw), a) = \delta(\delta^*(\delta^*(q, u), w), a) = \delta^*(\delta^*(q, u), wa) = \delta^*(\delta^*(q, u), v).$$

lo que cierra la inducción. ■

#### Transformaciones inducidas por palabras

Para cada  $w \in \Sigma^*$  definimos la transformación  $f_w : Q \rightarrow Q$ ,  $f_w(q) = \delta^*(q, w)$ . Por el caso base,  $f_\lambda(q) = \delta^*(q, \lambda) = q$ , luego  $f_\lambda = \text{id}_Q$ . Para  $u, v \in \Sigma^*$  y  $q \in Q$ , usando el Lema 0 y la convención  $f \cdot g = g \circ f$ :

$$f_{\{uv\}}(q) = \delta^*(q, uv) = \delta^*(\delta^*(q, u), v) = f_v(f_u(q)) = (f_u \cdot f_v)(q)$$

es decir,  $f_{\{uv\}} = f_u \cdot f_v$  (★). Esta identidad es **la pieza clave** de todo el desarrollo posterior: en la operación diagramática, la concatenación de palabras se traduce literalmente en concatenación de transformaciones, sin inversión de orden. En el código: `Transformation_of(u+v) == Transformation_of(u).then(Transformation_of(v))`.

### Definición, cerradura y existencia

Sea  $M(A) = \{ f_w : w \in \Sigma^* \} \subseteq Q^\Phi$ .

**Lema 1 (cerradura).**  $M(A)$  es cerrado bajo  $\cdot$ . Por  $(\star)$ , si  $f_u, f_v \in M(A)$  entonces  $f_u \cdot f_v = f_{\{uv\}} \in M(A)$ . ■

**Lema 2 (asociatividad).**  $\cdot$  es asociativa en  $Q^\Phi$  (y por tanto en  $M(A)$ ): para todo  $q$ ,  $((f \cdot g) \cdot h)(q) = h((f \cdot g)(q)) = h(g(f(q))) = (g \cdot h)(f(q)) = (f \cdot (g \cdot h))(q)$ . ■

**Lema 3 (identidad).**  $\text{id}_Q = f_\lambda \in M(A)$  es neutra a ambos lados:  $f_w \cdot f_\lambda = f_{\{w\lambda\}} = f_w = f_{\{\lambda w\}} = f_\lambda \cdot f_w$ , usando  $(\star)$  y  $w\lambda = w = \lambda w$  en  $\Sigma^*$ . ■

**Teorema 1.**  $(M(A), \cdot, \text{id}_Q)$  es un monoide finito y  $|M(A)| \leq |Q|^{|Q|}$ .

La demostración combina los Lemas 1–3; la cota se sigue de  $M(A) \subseteq Q^\Phi$  y  $|Q^\Phi| = |Q|^{|Q|}$ .

### 4.6 El homomorfismo natural $\varphi$

Definimos  $\varphi : \Sigma^* \rightarrow M(A)$  por  $\varphi(w) = f_w$ .

**Teorema 2 ( $\varphi$  es homomorfismo de monoides).**  $\varphi : (\Sigma^*, \cdot, \lambda) \rightarrow (M(A), \cdot, \text{id}_Q)$  cumple (i)  $\varphi(\lambda) = \text{id}_Q$ , y (ii)  $\varphi(uv) = \varphi(u) \cdot \varphi(v)$  para todo  $u, v \in \Sigma^*$ .

**Demostración.**

(i)  $\varphi(\lambda) = f_\lambda = \text{id}_Q$  por el caso base visto en la Sección 4.5. (ii) Por  $(\star)$ ,  $\varphi(uv) = f_{\{uv\}} = f_u \cdot f_v = \varphi(u) \cdot \varphi(v)$ . ■

**Teorema 3 ( $\varphi$  es sobreyectivo).**  $\text{Im}(\varphi) = M(A)$ , por construcción, ya que  $M(A) := \{ f_w : w \in \Sigma^* \} = \text{Im}(\varphi)$ . ■

Observación de implementación: la operación  $\cdot$  en  $M(A)$  corresponde exactamente al método `Transformation.then`; la igualdad  $\varphi(u).then(\varphi(v)) = \varphi(uv)$  se verifica empíricamente en `Homomorphism.verify_homomorphism` para todas las palabras hasta una longitud dada.

### 4.7 Núcleo de $\varphi$ , congruencia y clases de equivalencia

Definimos sobre  $\Sigma^*$  la relación  $u \sim v \Leftrightarrow \varphi(u) = \varphi(v) \Leftrightarrow f_u = f_v$ .

**Proposición 1.**  $\sim$  es una relación de equivalencia (reflexividad:  $f_u = f_u$ ; simetría:  $f_u = f_v \Rightarrow f_v = f_u$ ; transitividad:  $f_u = f_v \wedge f_v = f_w \Rightarrow f_u = f_w$ ). ■

**Proposición 2 ( $\sim$  es congruencia de monoide).** Si  $u \sim u'$  y  $v \sim v'$ , entonces  $uv \sim u'v'$ .

**Demostración.**

Por hipótesis  $f_u = f_{u'}$  y  $f_v = f_{v'}$ . Por  $(\star)$ ,  $f_{\{uv\}} = f_u \cdot f_v = f_{u'} \cdot f_{v'} = f_{\{u'v'\}}$ . Luego  $uv \sim u'v'$ . ■

Definimos  $\text{Ker}(\varphi) := \sim$ . Interpretación operativa: dos palabras  $u, v$  son  $\sim$ -equivalentes si y sólo si no se distinguen por el autómata desde ningún estado —para todo  $q \in Q$ ,  $\delta^*(q, u) = \delta^*(q, v)$ —, condición considerablemente más fuerte que  $\delta^*(q_0, u) = \delta^*(q_0, v)$ .

Por la Proposición 1, las clases  $[u] \sim = \{ v \in \Sigma^* : v \sim u \}$  particionan  $\Sigma^*$ . El cociente  $\Sigma^*/\text{Ker}(\varphi)$  hereda estructura de monoide con  $[u] \star [v] := [uv]$ , bien definida por la Proposición 2, con identidad  $[\lambda]$ .

## 4.8 Primer Teorema del Isomorfismo aplicado a las cadenas

**Teorema 4.** Existe un único isomorfismo de monoides  $\bar{\varphi} : \Sigma^*/\text{Ker}(\varphi) \rightarrow \text{Im}(\varphi)$  tal que  $\varphi = \bar{\varphi} \circ \pi$ , donde  $\pi : \Sigma^* \rightarrow \Sigma^*/\text{Ker}(\varphi)$  es la proyección canónica.

### Demostración.

Definimos  $\bar{\varphi} : \Sigma^*/\text{Ker}(\varphi) \rightarrow \text{Im}(\varphi)$  por  $\bar{\varphi}([w]) := \varphi(w) = f_w$ .

- **Buena definición.** Si  $[u] = [v]$ , entonces  $u \sim v$ , es decir  $\varphi(u) = \varphi(v)$ ; el valor  $\bar{\varphi}([w])$  no depende del representante elegido.
- **Homomorfismo.** Con  $[u] \cdot [v] := [uv]$  (bien definida por la Proposición 2),  $\bar{\varphi}([u] \cdot [v]) = \bar{\varphi}([uv]) = \varphi(uv) = \varphi(u) \cdot \varphi(v) = \bar{\varphi}([u]) \cdot \bar{\varphi}([v])$ , y  $\bar{\varphi}([\lambda]) = \text{id}_Q$ .
- **Inyectividad.** Si  $\bar{\varphi}([u]) = \bar{\varphi}([v])$ , entonces  $\varphi(u) = \varphi(v)$ , luego  $u \sim v$  y  $[u] = [v]$ .
- **Sobreyectividad sobre  $\text{Im}(\varphi)$ .** Trivial: si  $f \in \text{Im}(\varphi)$ , sea  $w$  cualquier preimagen;  $\bar{\varphi}([w]) = f$ .
- **Unicidad.** Cualquier  $\psi$  con  $\varphi = \psi \circ \pi$  debe cumplir  $\psi([w]) = \psi(\pi(w)) = \varphi(w) = \bar{\varphi}([w])$ .

■

Combinando con el Teorema 3 ( $\text{Im}(\varphi) = M(A)$ ):

**Corolario 1.**  $\Sigma^*/\text{Ker}(\varphi) \cong M(A)$  (isomorfismo de monoides).

Este corolario se verifica computacionalmente en `Homomorphism.verify_first_isomorphism`, comprobando: (a) toda transformación  $f \in M(A)$  aparece como  $\varphi(w)$  para alguna palabra de longitud  $\leq |M(A)|$ ; (b) las clases agrupan únicamente palabras con la misma imagen. La cota  $|M(A)|$  es suficiente porque cada nivel de la BFS añade al menos un elemento nuevo hasta agotar el monoide; por tanto el representante mínimo de cualquier  $f \in M(A)$  tiene longitud  $\leq |M(A)| - 1$ .

## 4.9 $\varphi$ y el grupo simétrico $S_n$ (profundización solicitada por el profesor)

Esta sección responde directamente a la observación del profesor de profundizar en la estructura de  $M(A)$ , la función  $\varphi$  y su relación con el grupo simétrico, explicando la composición en  $M(A)$  en términos de  $S_n$ .

**Permutación y grupo simétrico  $S_n$  [1].** dado un conjunto finito  $Q$  con  $|Q|=n$ , una permutación de  $Q$  es una función biyectiva  $Q \rightarrow Q$ . El conjunto de todas las permutaciones de  $Q$ , con la composición de funciones, forma un grupo llamado grupo simétrico y denotado  $S_n$  (o  $S_Q$  cuando

se quiere enfatizar el conjunto subyacente).  $|S_n| = n!$ , y  $S_n$  es, en general, no abeliano para  $n \geq 3$ .

Cada  $f_w \in M(A)$  es, por definición, una función  $Q \rightarrow Q$ , pero no toda función es una permutación: puede fallar la inyectividad. El siguiente teorema caracteriza exactamente cuándo  $M(A)$  es (isomorfo a) un subgrupo de  $S_n$ .

**Teorema 6 ( $M(A)$  subgrupo de  $S_n$ ).** Si para cada símbolo  $a \in \Sigma$  el generador  $g_a : Q \rightarrow Q$ ,  $g_a(q) = \delta(q, a)$ , es una biyección, entonces todo  $f_w \in M(A)$  es una permutación de  $Q$  y  $M(A)$  es un subgrupo de  $S_Q \cong S_n$  (con  $n = |Q|$ ).

### Demostración.

Toda palabra  $w = a_1 a_2 \dots a_k$  se descompone, por  $(\star)$ , como  $f_w = g_{\{a_1\}} \cdot g_{\{a_2\}} \cdot \dots \cdot g_{\{a_k\}}$ . La composición de funciones biyectivas es biyectiva, luego  $f_w$  es biyección para toda  $w$ , es decir  $f_w \in S_Q$ . Como  $M(A) \subseteq S_Q$  es cerrado (Lema 1), contiene la identidad (Lema 3) y  $S_Q$  es finito,  $M(A)$  es un submonoide de  $S_Q$  cerrado bajo composición de biyecciones; como además cada  $f_w \in S_Q$  tiene inverso en  $S_Q$  y ese inverso es también una potencia de  $f_w$  ( $S_Q$  es finito, así que todo elemento tiene orden finito, según [1]),  $M(A)$  contiene los inversos de sus elementos y es, por tanto, un subgrupo de  $S_Q$ .

■

Recíprocamente, si algún generador  $g_a$  no es inyectivo, entonces  $f_a$  no es una permutación (colapsa al menos dos estados), luego  $f_a \notin S_Q$  y  $M(A) \not\subseteq S_Q$ : en ese caso  $M(A)$  es un submonoide propio del monoide de transformaciones completo  $Q^\Phi = T_Q$  (que contiene a  $S_Q$  como subgrupo de sus elementos inversibles, pero es estrictamente más grande:  $|T_Q| = n^n \gg n! = |S_Q|$  para  $n \geq 3$ ).

### Ejemplo A — paridad de unos, $M(A) = S_2$

$Q = \{\text{Par}, \text{Impar}\}$ ,  $n = 2$ . El generador  $g_0 = \text{id}_Q$  y  $g_1 = (\text{Par} \text{ Impar})$ , la trasposición que intercambia los dos estados — la única permutación no trivial de  $S_2$ . Como ambos generadores son biyecciones, el Teorema 6 aplica:  $M(A) = \{\text{id}, g_1\} = S_2$ , y  $S_2 \cong \mathbb{Z}/2\mathbb{Z}$  por ser cíclico de orden 2, como se define en [1].

### Ejemplo B — módulo 3, $M(A)$ subgrupo cíclico de $S_3$

$Q = \{r_0, r_1, r_2\}$ ,  $n = 3$ .  $g_0 = \text{id}_Q$  y  $g_1 = (r_0 \ r_1 \ r_2)$ , un 3-ciclo con la notación de [1], elemento de  $S_3$  de orden 3. Como ambos generadores son biyecciones, el Teorema 6 aplica y  $M(A) = \{\text{id}, (r_0 \ r_1 \ r_2), (r_0 \ r_2 \ r_1)\}$  es el subgrupo cíclico  $\langle (r_0 \ r_1 \ r_2) \rangle$  de  $S_3$  generado por ese 3-ciclo — precisamente el subgrupo alternante  $A_3 \leq S_3$ . Nótese que  $M(A) \neq S_3$ :  $S_3$  tiene 6 elementos (incluye las 3 trasposiciones), mientras que  $M(A)$  sólo contiene los 3-ciclos y la identidad, porque ningún generador del alfabeto induce una trasposición. La composición en la tabla de Cayley que se muestra en la Sección 7.1 es exactamente la composición de ciclos en  $S_3$ :  $(r_0 \ r_1 \ r_2) \cdot (r_0 \ r_1 \ r_2) = (r_0 \ r_2 \ r_1) = (r_0 \ r_1 \ r_2)^{-1}$ , y  $(r_0 \ r_1 \ r_2) \cdot (r_0 \ r_2 \ r_1) = \text{id}$ .

### Ejemplo C — cadenas que terminan en “01”, $M(A) \not\subseteq S_n$

$Q = \{s_0, s_1, s_2\}$ ,  $n = 3$ . El generador  $g_0(s_0)=s_1$ ,  $g_0(s_1)=s_1$ ,  $g_0(s_2)=s_1$  colapsa los tres estados en uno solo ( $s_1$ ): no es inyectivo, luego  $g_0 \notin S_3$ . Por el Teorema 6 (contrarrecíproco),  $M(A)$  no puede ser subgrupo de  $S_3$ : en efecto, ninguno de sus 5 elementos salvo la identidad es invertible, y  $M(A)$  vive en el monoide de transformaciones completo  $T_Q = Q^Q$ , con  $|T_Q| = 3^3 = 27 \gg |S_3| = 6$ . Esta es precisamente la razón algebraica de que  $M(A)$  resulte aperiódico, como se detalla en la Sección 7.3: un elemento no inyectivo nunca puede tener inverso bajo composición, así que sus potencias eventualmente se estabilizan ( $x^k = x^{k+1}$ ) en lugar de ciclar de regreso a la identidad como ocurre en un grupo.

#### Ejemplo D — paridad conjunta de dos símbolos, $M(A) = S_2 \times S_2 \cong V_4$

$Q = \{(P,P), (P,I), (I,P), (I,I)\}$  codifica la paridad de dos contadores independientes,  $n = 4$ . El generador  $g_a$  intercambia la primera coordenada y  $g_b$  intercambia la segunda; ambos son biyecciones (trasposiciones que actúan sobre coordenadas distintas), luego el Teorema 6 aplica y  $M(A)$  es un subgrupo de  $S_4$ . Como  $g_a$  y  $g_b$  conmutan (afectan coordenadas independientes) y cada uno tiene orden 2,  $M(A) = \langle g_a, g_b \rangle$  es isomorfo al producto directo  $\mathbb{Z}/2\mathbb{Z} \times \mathbb{Z}/2\mathbb{Z}$ , tal como se define en [1], el grupo de Klein  $V_4$  — un subgrupo abeliano de  $S_4$  que no es cíclico, a diferencia de los Ejemplos A y B.

En síntesis: la pregunta “¿ $M(A)$  es un grupo?” tiene, gracias al Teorema 6, una respuesta puramente sintáctica sobre los generadores del alfabeto —sin necesidad de construir todo  $M(A)$  primero—, y cuando la respuesta es afirmativa,  $M(A)$  queda automáticamente identificado como subgrupo concreto de  $S_{|Q|}$ . Esta caracterización está implementada en `TransitionMonoid.is_group`, que primero decide inyectividad de cada generador y luego confirma la existencia de inversos en el monoide construido.

### 4.10 Invarianza respecto al AFD mínimo

El monoide  $M(A)$  se construye a partir de un AFD concreto  $A$ . Cabe preguntarse si depende sólo del lenguaje  $L(A)$  o del autómata particular. Introducimos, extendiendo a dos lados la relación de Myhill–Nerode presentada en [2]:

$$u \equiv_L v \Leftrightarrow \forall x,y \in \Sigma^*: xuy \in L \Leftrightarrow xvy \in L.$$

Definimos  $M(L) := \Sigma^*/\equiv_L$ , el monoide del lenguaje  $L$  (construcción propia; en [2] solo se trabaja la versión de un lado). Se cumple lo siguiente:

**Proposición 3.** Para un AFD  $A$  que reconozca  $L$ , con  $\sim$  la congruencia inducida por  $\phi_A$ , se tiene  $\sim \subseteq \equiv_L$  ( $u \sim v \Rightarrow u \equiv_L v$ ).

#### Demostración.

Si  $f_u = f_v$ , entonces para cualquier  $x,y \in \Sigma^*$ , usando el Lema 0:  $\delta^*(q_0, xuy) = \delta^*(f_u(\delta^*(q_0, x)), y) = \delta^*(f_v(\delta^*(q_0, x)), y) = \delta^*(q_0, xvy)$ . Como  $xuy$  y  $xvy$  llevan a  $q_0$  al mismo estado, pertenecen o no simultáneamente a  $L$ , luego  $u \equiv_L v$ . ■

**Corolario 2.** Existe un homomorfismo sobreyectivo natural  $\pi^*: M(A) \twoheadrightarrow M(L)$ ;  $M(L)$  es un cociente de  $M(A)$ . **Teorema 5 (caso de igualdad).** Si  $A_{\min}$  es el AFD mínimo que reconoce  $L$  (estados

= clases de Myhill–Nerode), entonces  $M(A_{\min}) \cong M(L)$ , porque en  $A_{\min}$  las congruencias  $\sim$  y  $\equiv_L$  coinciden.

Conclusión:  $M(A)$  es siempre tan fino o más fino que  $M(L(A))$ ; ambos coinciden exactamente cuando  $A$  es mínimo. Por la unicidad del AFD mínimo demostrada en [2],  $M(A_{\min})$  es un invariante del lenguaje  $L$ , no del autómata concreto. En este proyecto trabajamos directamente con  $M(A)$  porque se construye algorítmicamente sin minimización previa, ilustra completamente el aparato del Primer Teorema de Isomorfismo, y en los AFDs de ejemplo (ya mínimos)  $M(A) = M(L)$ .

## 5. Diseño de la solución y desarrollo realizado

### 5.1 Metodología

El proyecto sigue una metodología constructiva y verificable, en seis etapas:

- Modelado matemático: cada definición y teorema de la Sección 4 se expone con demostración detallada antes de programarse.
- Diseño orientado a objetos: cada concepto matemático se materializa en una clase Python con responsabilidad única (AFD, Transformation, TransitionMonoid, Homomorphism).
- Algoritmo BFS para construir  $M(A)$ : se inicia con la identidad y se aplica cada generador del alfabeto, deteniéndose cuando no aparecen transformaciones nuevas. Termina en a lo sumo  $|Q|^{|Q|}$  pasos, como se detalla en la Sección 6.
- Verificación empírica: para cada AFD de ejemplo se prueba computacionalmente que  $\phi(uv) = \phi(u) \star \phi(v)$ , que  $\sim$  es de equivalencia y que  $|\Sigma^*/\text{Ker}(\phi)| = |M(A)|$ .
- Pruebas unitarias (pytest) en todos los módulos: 149 pruebas en verde.
- Visualización del AFD (Graphviz), tabla de Cayley (matplotlib), reporte textual y sitio web interactivo (HTML/JS).

### 5.2 Arquitectura del sistema

El código está organizado en el paquete backend/, dimensionado para crecer hacia autómatas de pila (PDA) y máquinas de Turing en futuras iteraciones (slices); los módulos planos originales de la primera entrega se conservan en legacy/ como referencia histórica.

| Módulo                                  | Responsabilidad                                                                                                              |
|-----------------------------------------|------------------------------------------------------------------------------------------------------------------------------|
| <b>backend/models/dfa.py</b>            | Clase AFD: validación estructural, $\delta^*$ , ejecución, tabla de transición, minimización, producto, equivalencia.        |
| <b>backend/models/transformation.py</b> | Clase Transformation: $f : Q \rightarrow Q$ inmutable y hasheable, composición then/compose, propiedades algebraicas.        |
| <b>backend/models/nfa.py</b>            | AFN y AFN- $\lambda$ : $\lambda$ -closure, simulación, subset construction (en construcción / slice de lenguajes regulares). |

| Módulo                                            | Responsabilidad                                                                                                                                                                                                                                                                              |
|---------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>backend/algebra/transition_monoid.py</code> | Clase TransitionMonoid: construcción de $M(A)$ por BFS, tabla de Cayley, idempotentes, unidades, centro, <code>is_group</code> .                                                                                                                                                             |
| <code>backend/algebra/homomorphism.py</code>      | Clase Homomorphism: $\varphi$ , núcleo, cociente, <code>verify_homomorphism</code> , <code>verify_first_isomorphism</code> .                                                                                                                                                                 |
| <code>visualization.py</code>                     | Renderizado con Graphviz y matplotlib, generación de reportes de texto e imágenes.                                                                                                                                                                                                           |
| <code>main.py</code>                              | CLI interactivo (15 opciones) y subcomandos vía <code>argparse</code> ( <code>monoid</code> , <code>run</code> , <code>report</code> , <code>examples</code> , <code>from-regex</code> , <code>verify</code> , <code>infosheet</code> , <code>visualize</code> , <code>interactive</code> ). |
| <code>web/index.html</code>                       | Sitio estático de un solo archivo: sección Autómatas ( $\text{regex} \rightarrow \text{AFD mínimo}$ ) y sección Máquina de Turing, cada una con su panel de estructura algebraica de $M(A)$ calculado en vivo.                                                                               |

### 5.3 Decisiones de diseño relevantes

- **Transformation inmutable y hasheable.** Permite usarla como clave en dict/set durante la BFS sin pagar el costo de comparar diccionarios elemento a elemento; internamente almacena una tupla ordenada de pares  $(q, f(q))$ .
- **Orden de composición explícito.** Se distinguen los métodos `then` (orden de lectura,  $f.\text{then}(g) = f_{\{uv\}}$  cuando  $f=f_u$ ,  $g=f_v$ , según la convención descrita en la Sección 4.4) y `compose` (orden matemático clásico  $g \circ f$ ), evitando el error habitual de invertir el orden al concatenar.
- **BFS con representantes mínimos.** La cola se procesa en orden FIFO y los símbolos del alfabeto se aplican siempre en orden lexicográfico, lo que garantiza que la palabra representante de cada transformación sea la más corta y, entre las de igual longitud, la lexicográficamente menor.
- **Homomorphism.kernel truncado.** Devuelve una aproximación finita del núcleo (palabras con  $|w| \leq N$ ); para  $N \geq |M(A)|$  toda transformación ya se ha alcanzado, por lo que la verificación del Primer Teorema es exacta (Sección 4.8).
- **Visualización opcional.** Las funciones de `visualization.py` importan `graphviz/matplotlib` localmente, de modo que el resto del proyecto es utilizable en entornos sin librerías gráficas instaladas.

### 5.4 Interfaz web interactiva

Como complemento a la CLI, se desarrolló un sitio estático de un único archivo (`web/index.html`, sin backend ni instalación) con dos secciones. La sección Autómatas porta a JavaScript el parser de expresiones regulares, la construcción de AFN- $\lambda$ , la construcción de subconjuntos y la minimización descritas en [2]; los grafos se renderizan con la librería `viz.js` cargada por CDN. Para cada AFD generado, la página calcula en vivo, con exactamente el mismo aparato de la Sección 4: el tamaño de  $M(A)$  frente a la cota  $|Q|^{|Q|}$ , sus idempotentes, unidades  $U(M)$  y centro  $Z(M)$ , si es grupo (identificando  $\mathbb{Z}/n\mathbb{Z}$ ,  $V_4$ ,  $S_3$ , ..., según el Teorema 6 de la Sección 4.9), si es

abeliano, cíclico —con palabra generadora— o aperiódico. Incluye además un verificador de palabras con cinta animada, sincronizada con el resaltado del estado y la transición en el digrafo.

La segunda sección implementa el modelo completo de Máquina de Turing descrito en [2]: tupla  $(Q, q_0, F, \Sigma, \Gamma, \delta)$ , cinta bidireccional, blanco externo, desplazamientos  $\leftarrow/\rightarrow/-$ , con entrada por descripción del lenguaje ( $a^n b^n c^n$ ,  $\#a=\#b$ , palíndromos,  $ww$ , patrones con restricciones, o una expresión regular vía el teorema que relaciona AFD y Máquina de Turing presentado en [2]). Cuando el lenguaje descrito es regular, se construye  $M(A)$  del AFD mínimo equivalente con el mismo análisis de la sección anterior; si no lo es, la página explica por qué su monoide de transición sería infinito, según el criterio de no regularidad de [2], y la construcción finita no aplica.

## 6. Algoritmos, estructuras y modelos utilizados

### 6.1 Algoritmo de construcción de $M(A)$

```
Entrada: AFD  $A = (Q, \text{Sigma}, \text{delta}, q_0, F)$ 
Salida : conjunto  $M(A)$  subconjunto de  $Q^+Q$  y, para cada  $f$  en  $M(A)$ ,  $\text{rep}(f)$ .

1. Para cada  $a$  en  $\text{Sigma}$ , calcular el generador  $g\_a(q) = \text{delta}(q, a)$ .
2. Crear  $M \leftarrow \{\text{id}_Q\}$ ,  $\text{queue} \leftarrow [(\text{lambda}, \text{id}_Q)]$ ,  $\text{rep}(\text{id}_Q) \leftarrow \text{lambda}$ .
3. Mientras  $\text{queue}$  no este vacia:
  a. Desencolar  $(w, f)$ .
  b. Para cada  $a$  en  $\text{Sigma}$  (en orden lexicografico):
     $h \leftarrow f.\text{then}(g\_a)$       #  $h = f_{\{wa\}}$ 
    si  $h$  no esta en  $M$ :
       $M.\text{add}(h)$ ;  $\text{rep}(h) \leftarrow w+a$ 
       $\text{queue.append}((w+a, h))$ 
4. Devolver  $(M, \text{rep})$ .
```

Como la cola sólo crece cuando aparece una transformación nueva y  $|M(A)| \leq |Q|^{|Q|}$  (Teorema 1), el algoritmo termina. Sobre  $M(A)$  ya construido, `TransitionMonoid.is_group` aplica el criterio del Teorema 6 (Sección 4.9): decide si cada generador es biyectivo y, en caso afirmativo, confirma la existencia de inversos, identificando  $M(A)$  como subgrupo concreto de  $S_{\{|Q|\}}$  cuando corresponde.

### 6.2 Complejidad

El tiempo total del algoritmo BFS es  $O(|\Sigma| \cdot |M(A)| \cdot |Q|)$ : cada uno de los  $|M(A)|$  elementos descubiertos se procesa una vez, generando  $|\Sigma|$  transiciones nuevas, cada una calculada en  $O(|Q|)$  al aplicar la transformación sobre todos los estados. El espacio es  $O(|M(A)| \cdot |Q|)$  para almacenar las transformaciones. Como  $|M(A)| \leq |Q|^{|Q|}$ , el algoritmo es exponencial en el peor caso teórico, aunque en los ejemplos trabajados (Sección 7) el monoide observado es muchísimo más pequeño que la cota (p. ej. 5 frente a 27).

### 6.3 Estructuras de datos

- Transformation: tupla ordenada e inmutable de pares  $(q, f(q))$ , con `__hash__` y `__eq__` definidos sobre esa tupla — permite pertenencia en un set en tiempo esperado  $O(|Q|)$ .
- TransitionMonoid: conjunto (set) de Transformation, diccionario `rep : Transformation → str` para representantes mínimos, y la tabla de Cayley como diccionario  $(Transformation, Transformation) \rightarrow Transformation$ .
- Homomorphism: envuelve un AFD y expone `kernel(max_len)`, `verify_homomorphism(max_len)` y `verify_first_isomorphism()`, reutilizando TransitionMonoid internamente.

## 7. Resultados obtenidos

Se ejecutó el programa sobre los cuatro AFDs incluidos en `examples/`. Para cada uno, la suite de pruebas verifica que  $\varphi(uv) = \varphi(u).then(\varphi(v))$  para toda  $(u,v)$  con  $|u|, |v| \leq 4$ , que  $\sim$  es relación de equivalencia, y que  $|\Sigma^*/Ker(\varphi)| = |M(A)|$ , validando empíricamente el Corolario 1 (Sección 4.8).

### 7.1 Ejemplo 1: paridad de unos

$\Sigma=\{0,1\}$ ,  $Q=\{\text{Par}, \text{Impar}\}$ ,  $q_0=\text{Par}$ ,  $F=\{\text{Par}\}$ . Generadores:  $f_0 = \text{id}_Q$ ;  $f_1 = \text{intercambio}(\text{Par} \leftrightarrow \text{Impar})$ .  $M(A) = \{\text{id}, \text{swap}\} \cong \mathbb{Z}/2\mathbb{Z} = S_2$  (Teorema 6, Ejemplo A de la Sección 4.9).

|   | e | 1 |
|---|---|---|
| e | e | 1 |
| 1 | 1 | e |

Núcleo:  $u \sim v \Leftrightarrow |u|_1 \equiv |v|_1 \pmod{2}$ . Dos clases: palabras con número par de unos (incluye  $\lambda$ , 00, 11, 0110, ...) y con número impar (1, 10, 01, 111, ...). Primer Teorema:  $\Sigma^*/Ker(\varphi) = \{[\lambda], [1]\} \cong \mathbb{Z}/2\mathbb{Z} \cong M(A)$ .

### 7.2 Ejemplo 2: cantidad de unos $\equiv 0 \pmod{3}$

$\Sigma=\{0,1\}$ ,  $Q=\{r_0, r_1, r_2\}$ ,  $q_0=r_0$ ,  $F=\{r_0\}$ . El símbolo 0 no cambia el estado; el símbolo 1 lo incrementa módulo 3. Generadores:  $f_0=\text{id}_Q$ ;  $f_1=(r_0 \ r_1 \ r_2)$ , un 3-ciclo de  $S_3$ .  $M(A)=\{\text{id}, f_1, f_1^2\} \cong \mathbb{Z}/3\mathbb{Z}$ , subgrupo cíclico propio de  $S_3$  (Teorema 6, Ejemplo B de la Sección 4.9).

|    | e  | 1  | 11 |
|----|----|----|----|
| e  | e  | 1  | 11 |
| 1  | 1  | 11 | e  |
| 11 | 11 | e  | 1  |

Núcleo:  $u \sim v \Leftrightarrow |u|_1 \equiv |v|_1 \pmod{3}$ . Tres clases.

### 7.3 Ejemplo 3: cadenas que terminan en “01”

$\Sigma=\{0,1\}$ ,  $Q=\{s_0,s_1,s_2\}$ ,  $q_0=s_0$ ,  $F=\{s_2\}$ , con  $\delta(s_0,0)=s_1$ ,  $\delta(s_0,1)=s_0$ ,  $\delta(s_1,0)=s_1$ ,  $\delta(s_1,1)=s_2$ ,  $\delta(s_2,0)=s_1$ ,  $\delta(s_2,1)=s_0$ . Generadores:  $f_0=(s_0 \rightarrow s_1, s_1 \rightarrow s_1, s_2 \rightarrow s_1)$  — no inyectivo—;  $f_1=(s_0 \rightarrow s_0, s_1 \rightarrow s_2, s_2 \rightarrow s_0)$ . Por el Teorema 6 (contrarrecíproco),  $M(A) \not\subseteq S_3$  (Ejemplo C de la Sección 4.9).

Tras la BFS aparecen exactamente 5 transformaciones, con representantes mínimos  $\lambda$ , 0, 1, 01, 11. Su tabla de Cayley:

|    | e  | 0 | 1  | 01 | 11 |
|----|----|---|----|----|----|
| e  | e  | 0 | 1  | 01 | 11 |
| 0  | 0  | 0 | 01 | 01 | 11 |
| 1  | 1  | 0 | 11 | 01 | 11 |
| 01 | 01 | 0 | 11 | 01 | 11 |
| 11 | 11 | 0 | 11 | 01 | 11 |

**Núcleo (hasta  $|w| \leq 3$ )**

| Clase         | Palabras representativas      |
|---------------|-------------------------------|
| [ $\lambda$ ] | $\lambda$                     |
| [0]           | 0, 00, 10, 000, 010, 100, 110 |
| [1]           | 1                             |
| [01]          | 01, 001, 101                  |
| [11]          | 11, 011, 111                  |

$\Sigma^*/\text{Ker}(\varphi)$  tiene 5 elementos, en biyección con  $M(A)$ . La asimetría de la tabla confirma que el monoide es aperiódico:  $00 \sim 0$  pero  $11 \not\sim 1$ , y ningún elemento distinto de  $e$  tiene inverso.

### 7.4 Ejemplo 4: paridad conjunta de dos símbolos (grupo de Klein)

$\Sigma=\{a,b\}$ ,  $Q = \{(P,P),(P,I),(I,P),(I,I)\}$  codifica la paridad acumulada de  $a$  y de  $b$  por separado,  $q_0=(P,P)$ ,  $F=\{(P,P)\}$ . Generadores:  $g_a$  intercambia la primera coordenada,  $g_b$  intercambia la segunda; ambos son biyecciones que conmutan entre sí. Por el Teorema 6,  $M(A)$  es subgrupo de  $S_4$ ; como  $g_a$  y  $g_b$  tienen orden 2 y conmutan,  $M(A) \cong \mathbb{Z}/2\mathbb{Z} \times \mathbb{Z}/2\mathbb{Z} = V_4$ , el grupo de Klein (según [1], Ejemplo D de la Sección 4.9) — el único de los cuatro ejemplos cuyo grupo no es cíclico.

$|M(A)| = 4 = |Q|$ , y  $M(A)$  es abeliano pero no cíclico (todo elemento no identidad tiene orden 2). El núcleo tiene cuatro clases, correspondientes a las cuatro combinaciones de paridad de  $\#a$  y  $\#b$  en la palabra leída.

### 7.5 Tabla comparativa y verificación mediante pruebas

| AFD                    | Q | M(A) | Cota $ Q ^{ Q }$ | Conmutativo | Grupo | Estructura                              |
|------------------------|---|------|------------------|-------------|-------|-----------------------------------------|
| Paridad de unos        | 2 | 2    | 4                | sí          | sí    | $\cong \mathbb{Z}/2\mathbb{Z} = S_2$    |
| Módulo 3               | 3 | 3    | 27               | sí          | sí    | $\cong \mathbb{Z}/3\mathbb{Z} \leq S_3$ |
| Termina en "01"        | 3 | 5    | 27               | no          | no    | monoide aperiódico                      |
| Paridad conjunta (a,b) | 4 | 4    | 256              | sí          | sí    | $\cong V_4 \leq S_4$                    |

Los cuatro resultados están respaldados por 149 pruebas pytest en verde, que cubren: validación estructural y operaciones de AFD (minimización, intersección, equivalencia, contraejemplos); AFN con  $\lambda$ -transiciones ( $\lambda$ -closure, simulación, construcción de subconjuntos); álgebra de transformaciones (asociatividad, idempotencia, órbitas, ciclos); construcción de  $M(A)$  (cerradura, cota  $|Q|^{|Q|}$ , tabla de Cayley, idempotentes, unidades, aperiodicidad); propiedades de  $\varphi$  (homomorfismo, reflexividad/simetría/transitividad de  $\sim$ , Primer Teorema del Isomorfismo); parser de expresiones regulares y construcción de AFN- $\lambda$ ; verificación por equivalencia y por muestras; análisis de estructura de grupo ( $\mathbb{Z}/n\mathbb{Z}$ , Klein  $V_4$ ,  $S_3$ , ...); y generación de la hoja informativa pedagógica.

Salida típica del programa (python main.py monoid examples/mod3\_afd.json):

```

Monoide de transicion M(A) de Numero de 1s congruente con 0 mod 3
|Q|      = 3
|Sigma|   = 2
|M(A)|    = 3
Cota |Q|^|Q| = 27
Conmutativo = True
Grupo     = True

  | e  1  11
----+-----
e | e  1  11
1 | 1  11 e
11| 11 e  1

```

## 8. Limitaciones y posibles mejoras

### 8.1 Limitaciones actuales

- **Cota poco ajustada.** El Ejemplo 3 de la Sección 7.3 muestra que  $|Q|^{|Q|}$  dista mucho de ser una cota estrecha (5 frente a 27). En general, determinar  $|M(A)|$  exactamente sin construirlo es un problema no trivial: existen lenguajes regulares cuyo monoide de transición crece exponencialmente en  $|Q|$ .
- **Cociente truncado.** Computacionalmente no es posible exhibir las clases infinitas de  $\Sigma^*$ ; se exhiben hasta  $|w| \leq N$  con  $N \geq |M(A)|$ , lo cual es suficiente para verificar el isomorfismo

(la BFS garantiza que toda transformación aparece en a lo sumo  $|M(A)|$  pasos) pero no permite listar una clase de equivalencia completa.

- **Alcance a lenguajes regulares.** El análisis algebraico de  $M(A)$  sólo está definido para AFDs (lenguajes regulares). Los módulos `backend/models/pda.py` y `backend/models/turing.py` existen como esqueleto para autómatas de pila y máquinas de Turing, pero todavía no calculan una estructura algebraica análoga para lenguajes no regulares; la página web únicamente explica por qué el monoide de transición sería infinito en esos casos, según [2], sin construir un objeto algebraico equivalente.
- **Escalabilidad.** La representación de cada Transformation como tupla de pares  $(q, f(q))$  es adecuada para los AFDs de ejemplo ( $|Q| \leq 4$ ) pero no escala eficientemente a autómatas con cientos de estados, donde una representación matricial dispersa sería preferible.
- **Cobertura de familias de grupos.** Aunque el Teorema 6 de la Sección 4.9 caracteriza cuándo  $M(A)$  es grupo, el catálogo de ejemplos trabajados cubre sólo  $\mathbb{Z}/2\mathbb{Z}$ ,  $\mathbb{Z}/3\mathbb{Z}$  y  $V_4$ ; grupos no abelianos como  $S_3$  completo (con las tres trasposiciones) no aparecen como  $M(A)$  de ningún ejemplo incluido, aunque el marco teórico los admite.

## 9. Conclusiones

- Se logró un desarrollo riguroso del marco algebraico que vincula AFDs y monoides, con demostraciones completas de cerradura, asociatividad, identidad, homomorfismo, congruencia y Primer Teorema del Isomorfismo (Secciones 4.1 a 4.8), incluyendo la invarianza de  $M(A_{\min})$  respecto al lenguaje (Sección 4.10).
- Se caracterizó con precisión, mediante el Teorema 6 de la Sección 4.9, cuándo  $M(A)$  es subgrupo del grupo simétrico  $S_n$  —a partir únicamente de la biyectividad de los generadores del alfabeto— y se ilustró la composición en  $M(A)$  en términos de  $S_n$  para los cuatro ejemplos trabajados:  $S_2$ , un subgrupo cíclico de  $S_3$ , el escape total de  $S_3$  en el caso aperiódico, y  $V_4 \leq S_4$ .
- Se construyó una herramienta Python ejecutable, modular y probada (149/149 pruebas verdes) que materializa fielmente las definiciones matemáticas, más un sitio web interactivo de un solo archivo que calcula la misma estructura algebraica en vivo para cualquier expresión regular introducida por el usuario.
- Los cuatro ejemplos canónicos ilustran tanto el caso “grupo” cíclico (paridad, módulo 3), el caso “grupo” no cíclico (Klein  $V_4$ ), como el caso “monoide aperiódico” (termina en 01), mostrando que la estructura algebraica de  $M(A)$  capta información cualitativa genuina sobre el lenguaje reconocido.
- La integración entre Matemática Discreta II (estructuras algebraicas, relaciones de equivalencia, teoremas de isomorfismo, grupos simétricos) y Teoría de la Computación (AFDs, lenguajes regulares, Myhill–Nerode) queda explícita, demostrada y operativa tanto en el papel como en el código.

## 10. Referencias bibliográficas y técnicas

### 10.1 Fuentes bibliográficas

- [1] D. Saracino, Abstract Algebra: A First Course, 2.<sup>a</sup> ed. Long Grove, IL, EE. UU.: Waveland Press, 2008.
- [2] R. De Castro Korgi, Introducción a la Teoría de la Computación. Bogotá, Colombia: Departamento de Matemáticas, Universidad Nacional de Colombia, 2024.
- [3] Python Software Foundation, “The Python Language Reference, versión 3.10,” 2024. [En línea]. Disponible en: <https://www.python.org>
- [4] pytest Development Team, “pytest: helps you write better programs,” documentación oficial. [En línea]. Disponible en: <https://docs.pytest.org>
- [5] Graphviz, “Graphviz — Graph Visualization Software.” [En línea]. Disponible en: <https://graphviz.org>
- [6] I. D. Hunter, “Matplotlib: A 2D graphics environment,” Computing in Science & Engineering, vol. 9, n.º 3, pp. 90–95, 2007.
- [7] M. Daines, “viz-js: Graphviz en el navegador.” [En línea]. Disponible en: <https://github.com/mdaines/viz-js>
- [8] Grupo de desarrollo del proyecto, “syntactic-monoid,” repositorio GitHub, 2026. [En línea]. Disponible en: <https://github.com/Arax48/syntactic-monoid>